

# 반복을 자산으로, 외부를 연결로 — Skill·Command 자산화 + MCP 연동

커스텀 명령어·Skill 자산화 · Skill TOP10 · MCP로 파일·API 연결 · 리포트 Skill 전환

오후② · 학습목표 ②-b · 실습 → 개념 → 연결

반복은 Command-Skill로 «자산화»하고, 바깥은 MCP로 «연결»합니다

### ① 자산화 — Command-Skill

매번 다시 쓰던 지시를 파일로 굳혀 누가 불러도 같은 품질

### ② 연결 — MCP

파일·웹·API를 표준 규격으로 꽃아 최신 자료를 근거로

### ③ 안전 — 권한·검수

허용·차단을 최소로 정하고, «다음에도 맞는지»를 검수

쉽게 이해하기

Skill = «저장된 업무 매뉴얼», MCP = «만능 콘센트» — 3교시에 만든 작업공간에 매뉴얼과 바깥 창구를 놓습니다.

### 학습목표

- ☑ 기준을 파일로 남겨 프로토타입·SKILL을 만들고, «다음에도 맞는지» 검수할 수 있다
- ☑ Command-Skill의 차이를 알고, SKILL.md를 태그형 프롬프트로 설계할 수 있다
- ☑ MCP로 파일·API를 연결하고, 최소 권한으로 설정·점검할 수 있다

※ 재무 Dashboard를 "반복 가능한 자산(Skill)"으로 만들고 외부 데이터에 연결합니다.

진행 방식 · 실습 → 개념 → 연결 → 확장

X 기존 방식 — 개념부터

Command·Skill 정의 → MCP 프로토콜 설명 → 마지막에 따라하기

- 「내 업무에서 뭘 Skill로 만들지?」가 끝까지 안 잡힌다
- SKILL.md 문법은 외웠는데 **무엇을 적을지**를 모른다

✓ 오늘 방식 — 도구를 먼저 만들어 보고

프로토타입·SKILL 실습 → 「무엇을 파일로 남겼나」 해부 → Command·Skill 개념 → MCP 연결

- 「매번 다시 쓰던 걸 파일로 굳혔더니 되더라」를 먼저 겪는다
- Command·Skill·MCP가 방금 한 일과 1:1로 연결된다

60분 진행 흐름

① 실습

16분

프로토타입 · SKILL 만들기

② 해부

4분

무엇을 파일로 남겼나

③ 개념

12분

Command·Skill 개념 · 태그형 SKILL.md

④ 연결

12분

MCP 개념 · 연결 지시 · 최소 권한

⑤ 확장

16분

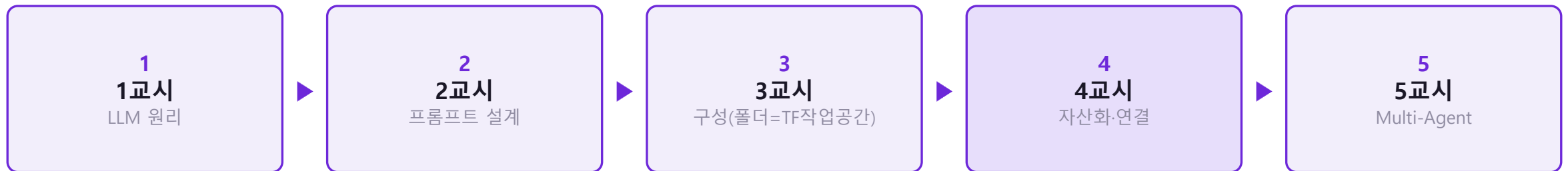
Yahoo 통합 프롬프트 → 내 반복 업무 Skill

쉽게 이해하기

1~3교시는 «결과물»을 만들었고, 4교시는 «만드는 도구»를 만듭니다. 앞선 산출물은 만들면 끝이었지만, 오늘의 산출물은 다음 사람이 이어서 쓰는 것입니다.

※ 실습 파일(입력 8개 · 산출물 샘플 3개)은 실습자료/4교시 폴더에 있습니다 — 참가자는 입력만 받고, 산출물은 실습 후 «정답지»로 공개합니다.

## 전체 구성 · 연계



### 3교시에서 한 일

폴더=TF작업공간 · CLAUDE.md(리더) · agent(실무자) · data(자료)로 "프로젝트"를 구성

### 4교시에서 더할 것

그 작업공간에 ① 반복 절차 자산(Command·Skill) ② 외부 연결(MCP)을 붙인다

### 쉽게 이해하기

3교시 = TF 작업공간을 만들었다면, 4교시 = 그 작업공간에 "업무 매뉴얼(Skill)과 바깥 창구(MCP)"를 놓는 단계입니다.

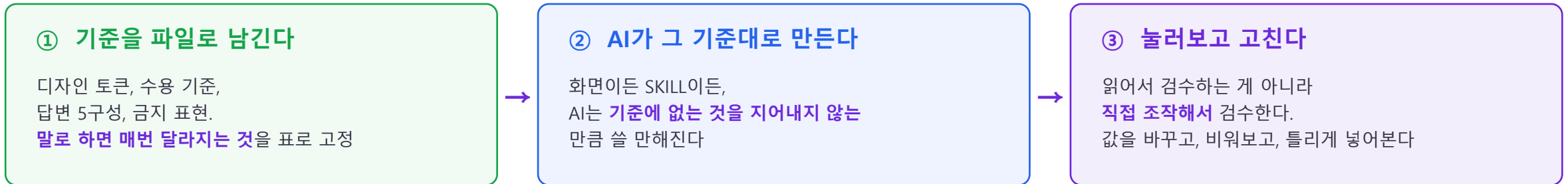
## ≡ 4교시가 바꾸는 것 — «결과물»에서 «도구»로

### 실습 도입 · 기준을 파일로 남긴다

앞선 세 교시의 산출물은 **만들고 나면 끝**이었습니다. 4교시의 산출물은 **다음 사람이 이어서 쓰는 것**입니다. 그래서 **무엇을 파일로 남길지**가 실습의 핵심이 됩니다.

| 구분       | 1~3교시 — 결과물 만들기   | 4교시 — 도구 만들기                         |
|----------|-------------------|--------------------------------------|
| 산출물의 수명  | 그 문서 한 건에서 끝난다    | 다음에도, 다른 사람도 그대로 쓴다                  |
| 재사용 방식   | 프롬프트를 매번 다시 쓴다    | 파일을 그대로 다시 실행한다                      |
| 1안이 하는 일 | 말로만 있던 요구를 문서로 정리 | 눌러볼 수 있는 <b>화면</b> 으로 만들어 설계 논의를 바꾼다 |
| 2안이 하는 일 | 한 번 잘 쓴 답변을 만든다   | 답변하는 <b>«방법»</b> 을 파일로 고정해 품질을 재현한다  |

### 도구를 만드는 3단계 — 이 순서를 벗어나면 «한 번만 잘 나온 결과»가 된다



### 이 실습을 성립시키는 3가지 원칙

- 요구는 문서에, 판단은 화면에**  
회의 30분보다 프로토타입 1개가 빠르다
- 프롬프트는 휘발되고 SKILL은 남는다**  
같은 일을 세 번 넘게 한다면 파일로 만들 때
- 둘 다 «초안»이다**  
SKILL 출력은 발송 전 초안 — 이 선을 분명히

## 실습 설계 · 프로토타입 → SKILL

1안은 눈에 보이는 것을 만들고, 2안은 눈에 보이지 않는 규칙을 만듭니다. 1안이 성취감이 크고 2안이 실무 효과가 큼니다 — 순서는 1안 → 2안을 권장합니다.

## 1안 · 프로토타입

필수

## 구매요청 화면 프로토타입 개발

바이브 코딩

DOCX · XLSX · PPTX · CSV → HTML

이럴 때 — 현업 요구를 IT에 설명하는 데 매번 실패하는 조직 — 기획·현업 담당자에게 효과적

## 2안 · SKILL

권장 / 시연 병행

## VOC 답변 SKILL 개발

반복 업무의 표준화

CSV · DOCX · PPTX · XLSX → SKILL.md + HTML

이럴 때 — 같은 질문에 사람마다 다르게 답하는 조직 — 헬프데스크·고객대응·총무에 효과적

| 비교 항목         | 1안 · 프로토타입 개발               | 2안 · SKILL 개발                 |
|---------------|-----------------------------|-------------------------------|
| 만드는 것         | 눌러볼 수 있는 화면 (HTML 1개)       | 답변하는 «방법» (SKILL.md + 실행 가이드) |
| 고정하는 기준       | 디자인 토큰 24개 · 수용 기준 10개      | 답변 5구성 · 금지표현 6개 · 품질 체크 10개  |
| AI에게 맡기는 판단   | 레이아웃과 상호작용 구현 방식            | 유형 분류 · 조항 매칭 · 에스컬레이션 여부     |
| AI가 하면 안 되는 것 | 토큰 밖의 색·크기를 임의로 쓰는 것        | 규정에 없는 내용을 지어내는 것             |
| 검수 방법         | 직접 눌러본다 (값 변경 · 빈칸 · 400px) | 근거를 원문과 대조한다 (조항 · 쪽수)        |
| 가장 흔한 실패      | 예쁘지만 아무 규칙도 지키지 않는 화면       | 그렇듯하지만 조항 번호가 틀린 답변           |

시간이 부족하면 2안의 SKILL 작성을 강사 시연으로 대체하고 참가자는 실행·검수만 합니다 — 마지막 「내 업무의 SKILL 후보」는 빠지지 않습니다.

## 실습 상세 · 난이도 중 · Claude Design + HTML

3교시에서 «진단»한 문제(병목·재작업·규정 위반)를 이번엔 «화면»으로 해결합니다. · 참가자는 **입력 4종만** 받고, 산출물은 **실습 후** 공개합니다.

## 📁 입력 — 자료 4종

DOCX

## 화면 요구사항 정의서 · 4쪽

개선 배경(3교시 진단) · RQ-01~RQ-10 · 수용 기준 · 검수 기준 5개

XLSX

## 화면목록 및 필드정의 · 5시트

화면 6개 · 필드 18개 · 상태 전이 7단계 · **디자인 토큰 24개**

PPTX

## 현행화면 이슈 · 6장

현행 화면 구조와 문제 3가지

CSV

## 코드값 정의 · 31행

드롭다운 코드 · 분류별 사양 라벨

## 📁 출력 — 동작하는 HTML 프로토타입

화면

SC-01 요청 작성 · SC-02 결재선 · SC-03 진행 현황 · SC-04 승인함

동작

금액 900 → 3,000 → 8,000 → 12,000 입력 시 결재선 8 → 9 → 10 → 11단계

차단

필수 사양 미입력 시 제출 비활성 + 무엇이 비었는지 표시

분기

표준계약서 선택 시 법무 3일 → 1일 (총 20일 → 18일)

대조

RQ-01~RQ-10 구현 위치 추적표

## 이 안에서 AI가 «판단»해야 하는 것

## 레이아웃 구현

요구사항의 배치 지시를 실제 화면 구조로 옮긴다

## 규칙 → 동작

금액·필수·계약서 유형 조건을 «눌러서 확인되는» 동작으로

## 추적표 작성

RQ 10개가 어디에 구현됐는지 스스로 대조한다

※ 요구사항·규정·VOC·설문은 모두 **가상 자료**입니다 — 실제 사내 규정과 VOC에는 개인정보·대외비가 포함되므로 가명 사본과 승인된 AI 환경을 쓰세요.

## 실습 실행 · 난이도 중 · Claude Design + HTML

### ① Claude Design — 뼈대

[자료들] 첨부 요구사항 DOCX · 필드정의 XLSX  
[기준] [디자인토큰] 시트의 색·글자 크기만 사용  
[배치] SC-01 입력폼 | 결재선 2단 · SC-03 단계 바+표 · SC-04 카드 목록  
[형태] 화면 4개를 캔버스에 스케치 — **동작 없이 배치만**

### ② HTML — 동작

[동작] ① 금액 입력 → 결재선 즉시 변경  
② 필수 사양 미입력 → 제출 차단 ③ 1,000만원 ↑ → 계약서 유형, 표준이면 간이검토  
[형태] 확정된 배치대로 **HTML 한 파일**  
[근거] RQ 번호 | 구현 위치 대조표 · 미구현은 「미구현」으로

## 프로토타입 프롬프트를 «여섯 칸»으로 분해하면

### ① 자료들

요구사항 DOCX  
필드정의 XLSX

### ② 기준

디자인 토큰 24개  
색·글자 크기 고정

### ③ 배치

화면 4개의 구조  
2단 · 단계 바 · 카드

### ④ 동작

결재선 계산  
제출 차단 · 분기

### ⑤ 형태

HTML 한 파일  
브라우저에서 열림

### ⑥ 근거

RQ 번호와  
구현 위치 추적표

## ✓ 검수 기준 5개 — 하나라도 미통과면 «미완성»

- ☐ 금액을 바꿔가며 눌렀을 때 결재선과 예상 소요일이 실제로 바뀌는가
- ☐ 필수 항목을 비웠을 때 제출이 막히고, 무엇이 비었는지 알려주는가
- ☐ 디자인 토큰 밖의 색·글자 크기가 쓰이지 않았는가
- ☐ 화면 폭 400px에서 레이아웃이 깨지지 않는가
- ☐ RQ 10개가 빠짐없이 추적표에 있는가 (미구현도 «미구현»이라고 적혀 있으면 통과)

🎯 **예쁜 화면이 아니라 «규칙을 지키는» 화면입니다** — 토큰 밖의 색을 쓰는 것이 가장 흔한 실패입니다.

## 실습 심화 · 배치 확정 → 동작 구현

화면을 만들 때 레이아웃 논의와 동작 구현을 한 번에 하면 둘 다 흐려집니다. 배치가 확정되기 전에 동작을 붙이면, **배치를 바꿀 때마다 동작을 다시** 만들게 됩니다.

## ① 캔버스 스케치

강사 시연

Claude Design으로 화면 4개의 **배치만** 잡는다

여러 화면을 한 캔버스에 늘어놓고 비교할 수 있다. 현업이 직접 요소를 옮기고 글자를 고칠 수 있어 「여기 말고 저기」라는 논의가 **말이 아니라 화면 위에서** 끝난다.

## ② 배치 확정

합의

현업·기획이 캔버스를 보고 **합의**한다

이 시점까지는 **동작이 없어도 된다**. 배치가 흔들리는 상태에서 동작을 만들면 작업이 두 배가 된다.

## ③ HTML 프로토타입

참가자 실습

확정된 배치에 **규칙과 동작**을 붙인다

결재선 계산, 필수 검증, 조건 분기처럼 **눌러봐야 확인되는 것**은 HTML에서 구현한다. 이때부터 검수 기준으로 판정할 수 있다.

## ④ 요구 변경 실험

반드시 직접

요구 **하나를 바꿔** 다시 생성한다

예: 「5,000만원 → 3,000만원으로 임원 승인 기준 변경」. **요구가 바뀌면 얼마나 빨리 반영되는지**가 프로토타입의 진짜 가치다.

쉽게 이해하기

①단계는 강사가 시연하고 참가자는 ③단계부터 직접 하는 편이 시간 대비 효과가 좋습니다. ④단계는 반드시 참가자가 직접 — 이 한 번의 경험이 「AI로 프로토타입을 만든다」의 의미를 가장 잘 전달합니다.

## 실습 상세 · 난이도 상 · SKILL.md + 실행 가이드

한 번 «답변하는» 것이 아니라 «답변하는 방법»을 만듭니다 — 규정·매뉴얼·과거 답변이 근거입니다. · 참가자는 **입력 4종만** 받고, 산출물은 **실습 후** 공개합니다.

### 입력 — 자료 4종

**CSV** VOC 접수이력 · 120건  
문의 · 답변 · 만족도 · 재문의 — **SKILL 설계의 근거**가 된다

**DOCX** 사내규정 발췌 · 4쪽  
제12·13·18·21·25·28·30조 (조·항·호 번호 포함)

**PPTX** 사용자 매뉴얼 · 7쪽  
슬라이드 번호 = 답변에서 인용하는 «쪽»

**XLSX** 답변품질 기준 · 6시트  
답변 5구성 · 유형별 SLA · 금지표현 6개 · 품질 체크 10개

### 출력 — SKILL.md + 실행 가이드

**SKILL.md** 참조 자료 · 6단계 절차 · 반드시 지킬 것 · 출력 형식 · 하지 않는 것

**실행 가이드** 왜 SKILL인가(데이터 근거) · 6단계 · SKILL.md 전문 · 실행 예시 3건

**실행 예시** 기본형 / 두 가지 문의 + 에스컬레이션 / 규정에 없는 질문

**효과** 재문의 22건 감소 · 처리시간 257시간 절감 (추정 — 산출 근거 표기)

## 이 안에서 AI가 «판단»해야 하는 것

**유형 분류**  
들어온 문의가 어느 유형인지 스스로 판정한다

**조항 매칭**  
규정 7개 조항 중 근거가 되는 조항을 골라 인용한다

**에스컬레이션**  
직접 답할 수 없는 건을 조건과 대상 부서까지 판정한다

※ 요구사항·규정·VOC·설문은 모두 **가상 자료**입니다 — 실제 사내 규정과 VOC에는 개인정보·대외비가 포함되므로 가명 사본과 승인된 AI 환경을 쓰세요.

## 실습 실행 · 난이도 상 · SKILL.md + 실행 가이드

## 구조화 프롬프트

[자료들] 첨부 VOC 이력 CSV · 사내규정 DOCX · 매뉴얼 PPTX · 품질기준 XLSX

[대상] 앞으로 들어올 IT 헬프데스크 문의의 답변 초안

[절차] SKILL.md에 ① 언제 이 SKILL을 쓰는지 ② 답변 작성 단계

[규칙] 근거 조·항 인용 · 금지표현 회피 · 에스컬레이션 판정 — 규정에 없으면 「확인 필요」

[형식] SKILL.md — 언제 쓰나 · 절차 · 반드시 지킬 것 · 출력 형식 · 하지 않는 것

[검증] 실제 문의 3건 적용 결과 + 품질 체크 10개 결과 — 미통과 항목도 숨기지 말 것

## SKILL 설계 프롬프트를 «여섯 칸»으로 분해하면

## ① 자료들

VOC·규정·매뉴얼  
품질기준 4종

## ② 대상

앞으로 들어올  
헬프데스크 문의

## ③ 절차

답변 작성 단계를  
6단계로

## ④ 규칙

조항 인용 · 금지표현  
에스컬레이션 판정

## ⑤ 형식

SKILL.md  
출력 형식 고정

## ⑥ 검증

문의 3건 실행 +  
품질 체크 10개

## ✓ 결과 검증 — 이것만은 확인한다

- ☐ 조항 번호가 정확한가 — 인용된 조·항을 규정 원문과 한 건 이상 대조
- ☐ 규정에 없는 질문에 답을 지어내지 않았는가 — 「확인 필요」로 남겼는지
- ☐ 에스컬레이션 대상을 스스로 판정했는가 — 조건·대상 부서까지
- ☐ 품질 체크 10개 결과를 스스로 출력했는가 — 불합격 항목을 숨기지 않았는지
- ☐ 같은 문의를 두 번 넣었을 때 같은 절차를 따르는가 (재현성)

🎯 데이터가 «규칙의 이유»를 말해줍니다 — 이 숫자를 스스로 찾아내 SKILL의 설계 근거로 삼았는지가 채점 포인트입니다.

## 실습 심화 · 설계 근거와 SKILL.md 6구성

## ① VOC 120건을 갈라 보면 — 「반드시 조항을 인용하라」의 근거

근거 조항을 인용한 답변

54건

만족도 4.5점

재문의율 4%

인용하지 않은 답변

66건

만족도 2.41점

재문의율 38%

그래서 SKILL의 규칙은 「모든 답변에 근거 조항을 인용한다」 — 규칙은 «취향»이 아니라 «데이터»에서 나옵니다.

## ② SKILL.md는 «여섯 덩어리»로 적는다 — 이 중 하나라도 빠지면 재현되지 않는다



## 언제 쓰나

이 SKILL이 발동하는 상황 — 「IT 헬프데스크 문의가 들어왔을 때」



## 절차 6단계

유형 분류 → 조항 검색 → 초안 → 금지표현 점검 → 에스컬레이션 판정 → 품질 체크



## 출력 형식

답변 5구성 (인사 · 확인 · 근거 · 조치 · 안내) 고정



## 참조 자료

규정 · 매뉴얼 · 품질기준 파일과 그 안의 어느 부분을 볼지



## 반드시 지킬 것

근거 조항 인용 · 금지표현 6개 회피 · SLA 명시



## 하지 않는 것

규정에 없는 답 지어내기 · 발송 — 발송은 사람이 한다

## ③ 그래서 나오는 효과 — 추정이지만 «산출 근거»를 함께 적는다

재문의 감소

22건

미인용 66건의 재문의율 38% → 4% 적용 시

처리시간 절감

257시간

건당 평균 단축 시간 × 연간 처리 건수

품질 편차

감소

답변 5구성·금지표현 고정으로 사람 간 차이 축소

## 실습 정리 · 휘발되는 것과 남는 것

같은 일을 세 번 넘게 했다면 그 방법을 파일로 만들 때가 된 것입니다. 오늘 만든 두 산출물은 성격이 다르지만 「기준을 파일로 고정한다」는 점이 같습니다.

|       | 프롬프트 (1~3교시)        | SKILL / 프로토타입 (4교시) |
|-------|---------------------|---------------------|
| 어디 있나 | 채팅창 안 — 대화가 끝나면 사라짐 | 파일 — 폴더에 남아 다시 실행됨  |
| 누가 쓰나 | 쓴 사람만               | 팀 전체 — 같은 파일을 공유    |
| 품질    | 쓸 때마다 조금씩 달라짐       | 절차가 고정되어 재현됨        |
| 개정    | 머릿속에서 고침            | 갱신 지점이 파일에 적혀 있음    |
| 검수    | 결과만 본다              | 절차와 근거를 함께 본다       |

### 🔄 전환 판단 — 셋 다 «예»면

- ☒ 같은 일을 월 3회 이상 하는가
- ☒ 판단 기준을 문장으로 적을 수 있는가
- ☒ 결과의 합격 기준이 있는가

→ SKILL로 만들 때가 됐습니다

## 오늘 실습이 남긴 «파일»과 그 이름

### HTML 프로토타입

요구를 «눌러볼 수 있는 형태»로 고정한 파일 — 설계 논의의 공용 언어

### SKILL.md

방법을 «절차와 규칙»으로 고정한 파일 — 모델이 스스로 불러 쓴다

### 실행 가이드

왜 그 규칙인지를 «데이터 근거»와 함께 남긴 문서 — 개정의 출발점

쉽게 이해하기

프롬프트는 휘발되고, SKILL은 남습니다 — 오늘 이후 강의는 「그 남는 것들에 어떤 이름이 붙어 있는가」입니다.

## 실습 정리 · 「다음에도 맞을 것인가」

앞선 세 교시가 「결과가 맞는가」를 봤다면, 4교시는 「다음에도 맞을 것인가」를 봅니다 — 한 번 잘 나온 것과 계속 잘 나오는 것은 다릅니다.

공통

## 기준 밖으로 나가지 않았는가

프로토타입은 **토큰 밖 색**, SKILL은 **규정 밖 내용** — 둘 다 「없는 것을 지어내는」 실패

공통

## 근거를 추적할 수 있는가

화면은 **RQ 번호**, 답변은 **조항 번호** — 근거 표기가 없으면 검수가 불가능

목업

## 값을 바꿔봤는가

읽지 말고 **누르세요** — 금액·분류·계약서 유형을 바꿔 결과가 달라지는지

목업

## 틀리게 넣어봤는가

필수를 비우고, 날짜를 어제로, 수량을 0으로 — **막히지 않으면 미완성**

목업

## 400px에서 열어봤는가

**결재자는 대개 이동 중에 승인한다**

목업

## 요구 변경이 반영되는가

요구 하나를 바꿔 재생성했을 때 **다른 부분이 깨지지 않는지**

SKILL

## 조항을 원문과 대조했는가

최소 한 건은 규정 원문에서 직접 확인 — **그럴듯한 오인용이 가장 위험**

SKILL

## «모른다»고 말하는가

규정에 없는 질문에 답을 만들지 않고 **「확인 필요」로 남기는지**

SKILL

## 두 번 실행했을 때 같은가

같은 문의를 두 번 넣어 절차와 결론이 같은지 — **다르면 규칙이 덜 적힌 것**

SKILL

## 갱신 지점이 적혀 있는가

규정이 바뀌면 무엇을 고칠지 — 없으면 **반년 뒤 틀린 답변**을 만들어낸다

⚠ **실습 데이터 주의** — 요구사항·규정·VOC·설문은 모두 **가상 자료**입니다. 실제 사내 규정과 VOC에는 **개인정보와 대외비**가 포함되므로 **가명 사본 + 승인된 AI 환경**을 사용하고, SKILL 산출물은 반드시 담당자 검토 후 발송하세요. 발송까지 자동화하지 않는 것이 원칙입니다.

## 검수를 «설계 단계»로 옮기는 한 줄

 자체 검수 지시

[출력] 결과와 함께 검수 기준별 «통과/미통과» 표 · [멈춤] 미통과 항목은 숨기지 말고 이유와 함께 남겨줘

## 실습 → 강의 브리지 · 체험에 이름 붙이기

실습에서 여러분은 **기준을 파일로 고정하고, AI가 그 기준으로 만들게** 했습니다. 그 행동 하나하나에 이름이 있고, 그것이 4교시의 강의 내용입니다.

매번 다시 쓰던 지시를 SKILL.md 파일로 굳혔다



모델이 «스스로» 불러 쓰는 재사용 자산

Skill

「구매요청 목업 만들어」를 이름으로 부르고 싶어졌다



사람이 «명시적으로» 부르는 단축 명령

Command

토큰·품질기준 시트를 «참조 자료»로 지정했다



자산이 무엇을 근거로 볼지 구조로 적는다

참조 자료 구조

규정·매뉴얼 파일을 매번 첨부해야 했다



매번 첨부 대신 «상시 연결»로 바꾼다

MCP

「발송은 사람이 한다」를 못 박았다



무엇까지 허용할지 경계를 정한다

권한(Scope)-안전

같은 문의를 두 번 넣어 «같은가»를 확인했다



재현되지 않으면 자산이 아니다

재현성 = 자산의 조건

쉽게 이해하기

«한 번 잘 시키는 법»에서 «계속 잘 나오게 만드는 법»으로 — 지금부터 그 이름들을 하나씩 설명합니다.

이름 붙이기 ① · Command-Skill 용어

| 용어               | 쉬운 설명                    | 비유       |
|------------------|--------------------------|----------|
| 커스텀 명령어(Command) | /로 부르는 나만의 단축 명령         | 단축키      |
| Skill            | 모델이 스스로 부르는 절차 자산        | 자격증 직원   |
| SKILL.md         | 스킬 1개 정의 파일(이름·설명·도구·절차) | 직무기술서    |
| frontmatter      | md 상단 --- 사이 메타(이름·설명)   | 이름표      |
| allowed-tools    | 스킬·커맨드가 쓸 수 있는 도구 목록     | 사용 허가 도구 |
| plugin           | 스킬·커맨드 묶음 배포 단위          | 앱 패키지    |
| 자산화              | 좋은 절차·프롬프트를 저장·재사용       | 템플릿 보관   |

Command=사용자가 부르는 단축키, Skill=모델이 알아서 꺼내 쓰는 기능 — 둘 다 "반복을 자산으로".



이름 붙이기 ② · MCP·연결·권한 용어

| 용어                      | 쉬운 설명                        | 비유          |
|-------------------------|------------------------------|-------------|
| MCP                     | AI와 외부 도구·데이터를 잇는 표준 연결      | 만능 콘센트      |
| Host·Client·Server      | 앱 / 연결 담당 / 도구 노출 프로세스       | 창구 구조       |
| Tools·Resources·Prompts | 실행 기능 / 자료 / 지시 템플릿          | 서버가 내주는 것   |
| Transport(stdio-http)   | 로컬 프로세스 / 원격 연결 방식           | 연결 통로       |
| 권한(scope)               | 무엇을 허용/차단할지                  | 출입증         |
| 파일시스템 / 공개 API          | 내 파일 연결 / 외부 데이터 연결          | 자료실 / 바깥 창구 |
| 커넥터                     | Claude Desktop에서 클릭으로 MCP 연결 | 플러그 꽂기      |

MCP는 "표준 콘센트" — 도구가 달라도 같은 방식으로 꽂아 씁니다. 강의 Ⅲ에서 그림으로 봅니다.

## 강의 I · 자산화 ① — 왜 반복 지시를 자산으로

## X 매번 다시 지시

- 같은 프롬프트를 매번 새로 작성
- 사람마다 결과·품질이 제각각
- 노하우가 개인 머릿속에만

## ✓ 명령어·Skill로 자산화

- 한 번 만들어 /명령 또는 자동 호출
- 누가 실행해도 같은 절차·품질
- 팀 자산으로 공유·재사용

## 쉽게 이해하기

프롬프트가 "한 번 쓰고 버리는 메모"라면, **Command·Skill**은 "**저장된 업무 매뉴얼**" — 반복 업무를 자산으로 바꿉니다.

## 강의 I · 자산화 ② — Command (사람이 부른다)

## 무엇인가

/로 부르는 나만의 단축 명령 — 저장된 프롬프트 절차

## 어디에 있나

.claude/commands/ 의 마크다운(.md) 파일

## 어떻게 부르나

/명령이름 [인자] — 사용자가 직접 호출(user-invoked)

## 예시 · .claude/commands/재무분석.md

```
---
description: 회사 재무 건전성 분석
allowed-tools: [Read]
---
$ARGUMENTS 회사의 재무 건전성을 부채비율·유동비율·이자보상배율로 [지표|값|판정] 표로 평가해줘.
```

이제 "/재무분석 삼성전자" 한 줄이면 저장된 절차가 그대로 실행됩니다 — 반복 지시 끝.

## 강의 I · 자산화 ③ — Skill (모델이 부른다)

### 커스텀 명령어 (Command)

- 사용자가 호출 (/명령)
- 저장된 프롬프트 절차 실행
- "이걸 실행해"를 단축

### Skill

- 모델이 스스로 호출 (자동)
- description 보고 필요 시 로드
- 이름·설명·허용도구·절차(SKILL.md)

#### 쉽게 이해하기

**Command** = "부르면 오는 단축키", **Skill** = "알아서 나서는 전문 기능" — 모델이 상황을 보고 적절한 Skill을 자동으로 씁니다.

## 강의 I · 자산화 ④ — Command·Skill의 구조

## Command 구조

- **/명령** → commands/이름.md
- frontmatter: description·allowed-tools
- 본문 = 프롬프트 템플릿(\$ARGUMENTS)
- → 호출 시 그대로 실행

## Skill 구조

- **SKILL.md** (스킬 폴더 1개)
- frontmatter: name·description·allowed-tools
- 본문 = 절차 + 번들(스크립트·자료)
- → 메타데이터 상시, 본문은 필요 시 로드

*Progressive Disclosure — Skill의 "설명"만 항상 떠 있다가, 필요할 때 본문·도구를 불러와 컨텍스트를 아키텍처.*

## 강의 I · 자산화 ⑤ — 어디에 두고 누가 쓰나

**어디에 두나**

.claude/commands/ · .claude/skills/ · 또는  
plugin으로 배포

**범위(Scope)**

project(팀 공유) · user(개인 전역)

**plugin**

스킬·커맨드 묶음을 marketplace로 설치·공유

**1****배치**

폴더/plugin에 추가

**2****인식**

설명(메타) 로드

**3****호출**

명령/자동 실행

**4****재사용**

팀·다음 업무

프로젝트에 두면 팀이 공유하고, user에 두면 내 모든 작업에서 쓸 수 있습니다.

## 강의 I · 자산화 ⑥ — 장점과 한계

## ✓ 장점

- 반복 작업 **자동화·표준화**
- 컨텍스트 절약(**필요 시 로드**)
- 도구·절차를 **안전하게 묶음**
- 팀 **협업·공유** 용이

## ✗ 단점 · 주의

- 초기 **설계·작성** 수고
- 설명(description)이 나쁘면 **호출 안 됨**
- 허용 도구 과다 시 **보안 위험**
- 너무 많으면 **관리 부담**

좋은 Skill의 관건은 "설명(description)" — 언제 쓰는지 명확해야 모델이 제때 호출합니다.

## 강의 Ⅱ · Skill 만들기 ① — 검증된 Skill 가져다 쓰기

| #  | Skill            | 무엇을 하나       | 재무 실습 활용         |
|----|------------------|--------------|------------------|
| 1  | pptx             | 프레젠테이션 생성    | 재무 요약 슬라이드       |
| 2  | xlsx             | 엑셀 집계·차트     | 재무 지표 표·차트       |
| 3  | docx             | 워드 문서 생성     | 재무 보고서 초안        |
| 4  | pdf              | PDF 읽기·생성·양식 | 공시 PDF 파싱·리포트 출력 |
| 5  | skill-creator    | 스킬 제작·최적화    | 재무 분석 Skill 만들기  |
| 6  | data-viz(차트)     | 데이터 시각화      | 건전성 지표 차트        |
| 7  | web-research     | 웹 리서치        | 기업·업종 조사         |
| 8  | report-generator | 정형 리포트 생성    | 재무 건전성 리포트       |
| 9  | code-review      | 코드 리뷰        | 개발 협업(참고)        |
| 10 | brand-guidelines | 브랜드 스타일 적용   | 사내 보고 양식 적용      |

※ 문서(pptx·xlsx·docx·pdf) 스킬이 특히 실무에서 강력합니다 — 재무 리포트를 바로 산출물로.

## 강의 Ⅱ · Skill 만들기 ② — 설치하는 3가지 방법

## ① Plugin 설치

marketplace add → install (스킬·커맨드 묶음)

## ② 폴더 배치

.claude/skills/<이름>/ 에 SKILL.md + 자료

## ③ 직접 작성

skill-creator로 SKILL.md 초안 생성·개선

## SKILL.md 상단(frontmatter) 예시

```
---
name: 재무건전성분석
description: 재무제표로 건전성을 표로
  평가할 때 사용. 부채/유동/이자보상배율.
allowed-tools: [Read, xlsx]
---
(본문: 분석 절차 1) 2) 3) ...)
```

## • 설치 프롬프트(예)

- "skill-creator로 재무건전성분석 스킬을 만들어줘"
- "xlsx 스킬로 이 지표를 표·차트로 만들어줘"
- "이 폴더에 재무 리포트 스킬을 추가해줘"
- → 등록 후엔 모델이 상황 보고 자동 호출

비개발자는 ① Plugin 설치가 가장 쉽고, 나만의 절차는 ③ skill-creator로 만들면 됩니다.

## 강의 Ⅱ · Skill 만들기 ③ — 반복 작업 식별

## STEP 1

## 매번 반복하는 "절차"를 하나 고릅니다

- 매주/매번 같은 순서로 하는 업무를 찾는다
- 재무 리포트: 수집→분석→표·차트→요약
- "이건 매번 똑같다" 싶으면 Skill 후보

좋은 후보 = 자주·반복·절차가 분명한 일. 재무 건전성 분석이 딱 맞습니다.

## 강의 Ⅱ · Skill 만들기 ④ — SKILL.md로 정의

### STEP 2

### 이름·설명·허용도구·절차를 SKILL.md에 적습니다

- name-description(언제 쓰는지)을 명확히
- allowed-tools에 필요한 도구만
- 본문에 1)2)3) 절차를 순서대로

*description이 핵심 — "재무제표로 건전성 평가할 때"처럼 상황을 분명히 적어야 자동 호출됩니다.*

## 강의 Ⅱ · Skill 만들기 ⑤ — 저장·등록 (Scope)

## STEP 3

## 프로젝트/개인 범위에 저장해 등록합니다

- .claude/skills/ 에 폴더로 저장
- project=팀 공유, user=내 전역
- 필요 자료(양식·예시)도 함께 번들

팀이 같이 쓰려면 *project* 범위 — 프로젝트를 여는 누구나 같은 *Skill*을 사용합니다.

## 강의 Ⅱ · Skill 만들기 ⑥ — 호출·재사용

## STEP 4

## 이제 자동 호출되거나 명령으로 재사용합니다

- 모델이 상황 보고 스스로 호출
- 또는 커스텀 명령(/재무분석)으로 호출
- 새 회사·새 기간에도 그대로 재사용

한 번 만든 Skill은 다음 주에도, 다른 회사 분석에도 그대로 — 자산이 됩니다.

## 강의 Ⅱ · Skill 만들기 ⑦ — SKILL.md를 태그형으로

SKILL.md를 **문단으로 쓰면** 언제 부르는지·하지 말아야 할 것이 묻힙니다. **칸 이름을 태그로 붙이면** 모델도 사람도 빠진 칸을 바로 찾습니다.

### ✗ 서술형 — Skill 설명을 한 문단에

재무 분석할 때 쓰는 스킬이야. 재무제표 읽고 지표를 계산해서 판정하고 표로 만들어줘. 필요한 도구는 알아서 쓰고, 결과는 보기 좋게 정리해줘.

- 언제 부르는지가 흐려 **자동 호출이 안 된다**
- 「도구는 알아서」 — **허용 도구가 과다해진다**
- 멈출 조건이 없어 **지어내기를 막지 못한다**

### ✓ 구조형 — SKILL.md를 태그 칸으로

**[name]** 재무건전성분석  
**[description]** 재무제표로 건전성을 표로 평가할 때 사용  
**[allowed-tools]** Read, xlsx  
**[절차]** 1) data 재무제표 읽기 2) 지표 계산 3) CLAUDE.md 기준 판정 4) 표·차트  
**[출력]** [지표 | 값 | 업종평균 | 판정] · 수치마다 연도·출처  
**[하지 않는 것]** 없는 수치 추정 · 발송 — 자료가 없으면 멈추고 요청



## 구조형 작성 규칙 4 — Claude Code로 설계 ①~④와 확장 실습이 모두 이 형식입니다

### ① description = 호출 조건

「무엇을 할 때 쓰는지」를 적어야 모델이 제 때 부른다

### ② allowed-tools는 최소

필요한 도구만 — 권한 과다는 보안 위험

### ③ 절차는 번호 + 입출력

단계마다 무엇을 받고 무엇을 내는지 적는다

### ④ [하지 않는 것]에 멈춤

자료가 없거나 규정 밖이면 지어내지 말고 멈춘다

### 쉽게 이해하기

SKILL.md는 «직무기술서»입니다 — **칸이 비어 있지 않아야** 누가 불러도 같은 방식으로 일합니다.

## 강의 Ⅱ · Skill 만들기 ⑧ — 초안 생성 (태그형 프롬프트)

STEP 1

반복 업무를 Skill 초안으로 만들게 합니다

### ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 AI Agent Skill 설계 전문가야.

[맥락] 우리 팀은 매주 재무 건전성 리포트를 ① 재무제표 수집 → ② 지표 계산 → ③ 판정 → ④ 표·요약 순서로 만든다.

[요청] 이 절차를 **재사용 가능한 Skill 초안**으로 만들어줘.

[출력] SKILL.md — name · **description(언제 쓰는지)** · allowed-tools · 본문 절차(단계마다 입력·출력)

[검증] 이 Skill이 자동 호출될 상황 3가지 예시 · 절차에 빠진 입력이 있으면 [ ]로 비워 질문

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

따라하기 포인트: 막막할 땐 초안을 Claude에게 맡기고, 사람은 방향과 검수에 집중합니다.

## 강의 Ⅱ · Skill 만들기 ⑨ — 설명·권한 다듬기

### STEP 2

이름·설명·허용 도구를 정확히 다듬습니다

#### ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 AI Agent Skill 설계 전문가야.

[맥락] 방금 만든 재무건전성분석 SKILL.md · 기준: name=용도가 한눈에 · description=**호출 상황** · allowed-tools=**최소**(Read, xlsx)

[요청] 실제로 쓸 수 있게 다듬어줘.

[출력] [항목 | 전 | 후 | 왜 바꿨나] 표

[검증] description만 보고 언제 쓰는 Skill인지 알 수 있는지 스스로 점검

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

따라하기 포인트: description이 나쁘면 스킬이 있어도 호출되지 않습니다 — 가장 중요한 한 줄.

## 강의 Ⅱ · Skill 만들기 ⑩ — 절차·출력 고정

### STEP 3

본문에 실행 절차와 출력 형식을 고정합니다

#### ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 AI Agent Skill 설계 전문가야.

[맥락] 절차 1) data 재무제표 읽기(없으면 요청) 2) 부채·유동·이자보상배율·영업이익률 계산  
3) CLAUDE.md 기준으로 위험/주의/보통 판정 4) xlsx로 [지표|값|업종평균|판정] 표·차트

[요청] 재무건전성분석 SKILL.md 본문에 이 실행 절차를 확정해줘.

[출력] 모든 수치에 연도·출처 · 위험 판정 지표는 원인 1줄

[검증] 데이터가 일부 없을 때의 예외 처리를 절차에 포함 · **없는 값은 추정하지 말 것**

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

따라하기 포인트: 절차와 출력 형식을 고정할수록 누가 실행해도 같은 품질이 나옵니다.

## 강의 표 · Skill 만들기 ⑪ — 실행 테스트·개선

STEP 4

실제로 실행해 보고 SKILL.md를 개선합니다

✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 AI Agent Skill 품질 검수자야.

[맥락] 검수 관점 — 4단계 수행 · 표 형식 일치 · 연도·출처 표기 · 위험 판정의 원인 설명

[요청] 재무건전성분석 Skill을 샘플 재무제표로 실행해줘.

[출력] [검수 항목 | 통과 | 문제점 | 수정 제안] 표

[검증] 미통과는 SKILL.md의 **어느 줄을 어떻게** 고칠지 제시 → 수정본 적용 후 재실행해 전/후 비교

1 Prompt

2 Context

3 Harness

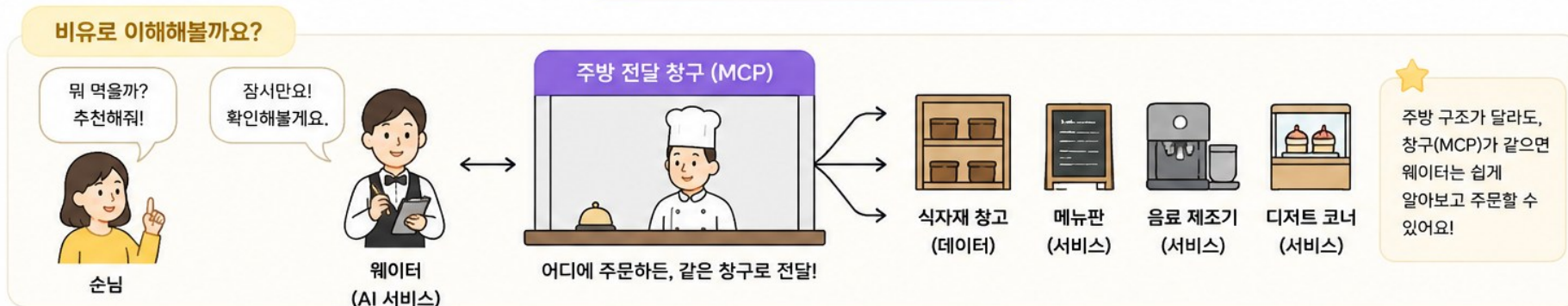
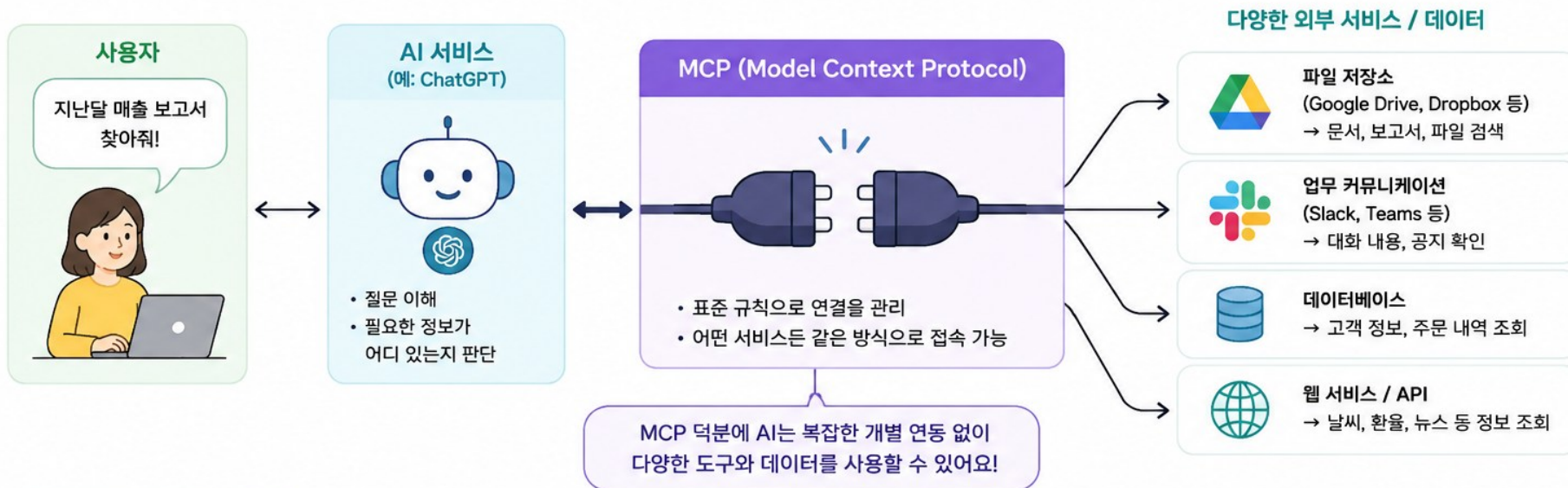
4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

따라하기 포인트: Skill도 코드처럼 테스트·개선 — 한 번에 완벽하지 않아도 됩니다(Loop).

MCP는 AI가 다양한 외부 서비스나 데이터에 쉽게 접근할 수 있도록 도와주는 ‘표준 연결 방식’입니다.



**정리하자면!**

MCP는 AI와 다양한 도구/데이터를 연결해 주는 ‘**공통 콘센트**’ 같은 역할을 합니다.

## 강의 Ⅲ · MCP ② — 표준 콘센트 비유

 MCP 없으면

- AI는 자기가 아는 것만 말함
- 최신 뉴스·실시간 금리 모름
- "손"이 없는 두뇌

 MCP 있으면

- 웹 검색·파일·API에 연결
- 최신·외부 데이터를 근거로
- 도구를 쓰는 "일하는 AI"

## 쉽게 이해하기

MCP = 만능 콘센트 — Claude Code라는 가전에 "웹 검색"·"내 파일"·"금융 데이터" 같은 플러그를 꽂아 능력을 확장합니다.

## 강의 Ⅲ · MCP ③ — Tools · Resources · Prompts

### 무엇인가

LLM을 외부 도구·데이터에 잇는 개방형 표준  
(Anthropic)

### 왜 표준인가

도구마다 다른 방식이 아니라 "하나의 규격"  
— 한 번 만들면 어디서나

### 무엇을 잇나

파일·웹·DB·사내 시스템 등 외부 자원

## MCP가 노출하는 3가지 — Tools · Resources · Prompts



### Tools

모델이 호출하는 "동작"(검색·조화·실행)



### Resources

모델이 참고하는 "자료"(파일·데이터)



### Prompts

재사용 가능한 "지시 템플릿"

*MCP = "AI와 외부 세계를 잇는 공용 규격" — USB-C처럼 어떤 도구든 같은 방식으로 꽂습니다.*

## 강의 Ⅲ · MCP ④ — 장점과 한계

## ✓ 장점

- 실시간·외부 데이터를 근거로 답변
- 검색·조회 등 행동까지 수행
- 지식 시점 한계·환각 감소
- 사내 시스템·API와 연결 확장

## ✗ 단점 · 주의

- 서버 설정·관리 필요(복잡도 ↑)
- 권한 과다 시 보안 위험
- 외부 호출로 지연·비용 발생
- 결과가 커지면 컨텍스트 부담·서버 의존

장점이 크지만 "무엇을 허용할지(권한)·무엇을 연결할지"를 신중히 정해야 안전합니다.

## 강의 Ⅲ · MCP ⑤ — 대표 서버 TOP 10

| #  | MCP 서버              | 무엇을 하나         | 재무 실습 활용        |
|----|---------------------|----------------|-----------------|
| 1  | Filesystem          | 로컬 파일 읽기/쓰기    | data 폴더 재무제표 읽기 |
| 2  | Fetch               | 웹 페이지·URL 가져오기 | 공시·기사 원문 수집     |
| 3  | Web Search          | 웹 검색(브레이브 등)   | 기업 최신 뉴스 검색     |
| 4  | Playwright          | 브라우저 자동화       | 웹 화면에서 데이터 추출   |
| 5  | Database(PG·SQLite) | DB 조회          | 사내 재무 DB 조회     |
| 6  | Google Drive        | 문서·시트 접근       | 보고서 자료 불러오기     |
| 7  | GitHub              | 저장소·이슈·코드      | 개발 협업(참고)       |
| 8  | Slack               | 메시지·알림         | 분석 결과 공유        |
| 9  | Memory              | 정보 기억 저장       | 분석 맥락 유지        |
| 10 | context7            | 최신 라이브러리 문서    | 개발 참고(참고)       |

※ 서버 생태계는 계속 늘어납니다. 재무 실습엔 Web Search·Fetch·Filesystem·Database가 특히 유용합니다.

## 강의 Ⅲ · MCP ⑥ — 부서별 활용

| #  | MCP 서버                    | 무엇을 연결      | 주 사용 부서  | 업무 활용 예               |
|----|---------------------------|-------------|----------|-----------------------|
| 1  | Filesystem                | 사내 문서·자료 폴더 | 전 부서     | 규정·양식·과거 보고서 읽어 초안 작성 |
| 2  | Database (SQL)            | 사내 DB 조회    | 재무·회계·구매 | 원장·전표·발주 내역 집계        |
| 3  | Excel / Spreadsheet       | 표·집계·차트     | 재무·회계·총무 | 예산 대비 실적 표·차트 자동 생성   |
| 4  | Web Search                | 최신 시장·정책 정보 | 경영혁신·구매  | 벤치마킹·원자재 시세·규제 동향     |
| 5  | Fetch (URL)               | 지정 웹페이지 원문  | 경영혁신·재무  | 공시·IR 자료·기사 원문 수집     |
| 6  | Google Drive / SharePoint | 문서 저장소      | 총무·인사    | 품의서·계약서·인사 서식 조회      |
| 7  | Slack / Teams             | 업무 커뮤니케이션   | 전 부서     | 공지 작성·회의 결과 요약 공유     |
| 8  | PDF                       | 증빙·계약서 파싱   | 회계·구매    | 세금계산서·계약 조건 추출        |
| 9  | Playwright                | 웹 화면 자동 조작  | 구매·경영혁신  | 공급사 포털 조회·경쟁사 화면 캡처   |
| 10 | Memory / Knowledge        | 사내 지식 축적    | 인사·경영혁신  | 제도·FAQ를 기억해 일관 응대     |

※ 부서별 대표 조합 — 재무·회계: ①②③⑧ / 구매: ②④⑧⑨ / 총무·인사: ①③⑥⑩ / 경영혁신: ④⑤⑨⑩ · 실제 연결은 보안 정책에 따라 IT 승인 필요.

## 강의 Ⅲ · MCP ⑦ — 연결하는 3가지 방법

### ① Desktop 커넥터

Claude Desktop 설정 → 커넥터/MCP에서 추가  
(비개발자 친화)

### ② 설정 파일

claude\_desktop\_config.json 의 mcpServers 에  
등록

### ③ CLI 추가

claude mcp add <이름> ... (Claude Code 사용  
시)

#### 설치 예시 (설정 파일)

```
"mcpServers": {
  "filesystem": {
    "command": "npx",
    "args": ["-y",
      "@modelcontextprotocol/server-filesystem",
      "C:/dev/재무건전성-분석"]
  }
}
```

#### • 설치 프롬프트(예)

- "웹 검색 MCP로 삼성전자 최근 뉴스 가져와"
- "filesystem으로 data의 재무제표 읽어줘"
- "연결된 MCP 서버 목록을 보여줘"
- → 등록 후엔 **평소처럼 말로 지시**

비개발자는 ① Desktop 커넥터로 클릭 연결이 가장 쉽습니다. 등록만 하면 이후엔 자연어로 사용합니다.

## 강의 IV · 연결 ① — 연결 지시를 태그로

「파일이랑 인터넷 연결해줘」로는 어디까지 열지가 정해지지 않습니다. 연결 지시는 **대상 · 권한 · 확인**을 칸으로 나눠 씁니다.

## X 막연한 연결 지시

파일이랑 인터넷 연결해서 필요한 거 다 가져올 수 있게 해줘.

- 어느 폴더·어느 API인지 **범위가 없다**
- 읽기·쓰기·삭제 중 **무엇을 허용할지** 모른다
- 연결됐는지 **확인할 방법**이 없다



## ✓ 구조형 — 연결 지시를 태그 칸으로

**[역할]** 너는 AI 도구 연동 담당 엔지니어야.  
**[맥락]** 대상 폴더 C:\dev\wfdew\재무건전성-분석 · 외부 기준금리·환율 API  
**[요청]** filesystem·API MCP를 연결해줘.  
**[출력]** 설정 단계 + [도구 | 허용 | 차단] 표  
**[검증]** data 파일 1개를 실제로 읽어 요약 · 차단 폴더 접근이 막히는지 확인

## 연결 지시 작성 규칙 4 — 연결 ①~④ 프롬프트가 모두 이 형식입니다

## ① 대상은 경로·범위까지

폴더·API를 이름으로 — 「전부」는 금지

## ② 허용과 차단을 함께

읽기·쓰기·삭제를 도구별로 적는다

## ③ 민감 파일은 전면 차단

.env·개인정보 파일은 처음부터 막는다

## ④ 실제 동작으로 확인

「연결됨」 메시지가 아니라 읽기·차단을 직접 시험

쉽게 이해하기

연결은 «출입증 발급»입니다 — 어느 문을 열고 어느 문은 잠가 둘지 먼저 정해야 안전합니다.

## 강의 IV · 연결 ② — 강사 시연 관찰

### ▶ 실습

강사가 뉴스·금리·환율 데이터를 MCP로 불러오는 과정을 관찰합니다

- 웹 검색 MCP로 대상 기업의 최근 뉴스를 가져오는 모습
- 금융 지표(금리·환율)를 외부에서 불러오는 과정
- 불러온 데이터가 분석의 "근거"로 붙는 것을 확인

### 관찰 포인트 — 이것을 보세요

- ☑ MCP가 "무엇을·왜 연결하는지" 화면에서 짚을 수 있다
- ☑ 외부 데이터가 분석 결과의 근거가 됨을 이해한다
- ☑ 연결 없이는 최신·외부 정보를 못 쓴다는 점을 안다

다음 5교시: 이 외부 데이터를 여러 에이전트가 나눠 분석해 Dashboard로 만듭니다.

## 강의 IV · 연결 ③ — 파일시스템 연결

## STEP 1

내 폴더의 자료를 Skill이 읽도록 연결합니다

## ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 AI 도구 연동 담당 엔지니어야.

[맥락] 폴더 C:\dev\wfde\재무건전성-분석 · 읽을 파일 data\손익계산서·대차대조표(csv) · 사용 주체 재무건전성분석 Skill

[요청] filesystem MCP로 이 폴더를 연결해줘.

[출력] 연결 설정 단계 + data 파일 목록 표

[검증] 손익계산서 매출·영업이익 한 줄 요약으로 **실제 읽기** 확인 · 폴더가 없으면 만들지 말고 알려줘

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

연결 포인트: Skill이 "무엇을 근거로" 일할지 자료 소스를 지정하는 단계입니다.

## 강의 IV · 연결 ④ — 공개 API 연결

## STEP 2

## 외부 시세·거시 지표를 가져오도록 연결합니다

## ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 AI 도구 연동 담당 엔지니어야.

[맥락] 필요 데이터 — 한국 기준금리(최신) · 원/달러 환율(최신) · 대상 기업 최근 1개월 뉴스 5건

[요청] 외부 데이터를 가져오는 MCP를 연결해 리스크 평가에 쓰게 해줘.

[출력] [지표 | 값 | 기준일 | 출처] 표 · 뉴스는 [제목 | 날짜 | 재무 영향 한 줄]

[검증] 출처·조회 시점 표기 확인 · 값이 비면 다른 출처로 재조회, 끝내 없으면 「조회 실패」로 표기

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

연결 포인트: 외부 API = 최신 거시 지표 — 리스크 평가의 근거가 됩니다.

## 강의 IV · 연결 ⑤ — 권한 최소화

## STEP 3

허용·차단 범위를 최소 권한으로 설정합니다

## ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 보안을 담당하는 AI 도구 관리자야.

[맥락] 정책 — data/ 읽기만 · output/만 쓰기(그 외 쓰기·삭제 금지) · 외부 API 조회(GET)만 · .env·개인정보 파일 전면 차단

[요청] 연결된 도구의 권한을 최소 권한 원칙으로 설정해줘.

[출력] [도구 | 허용 | 차단 | 이유] 표 + 어디에 어떻게 적용하는지 단계

[검증] 차단 대상에 실제로 접근이 안 되는지 확인하는 방법까지

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

연결 포인트: 권한 최소화 = 안전 — 필요한 것만 허용하고 위험한 것은 명시적으로 차단.

## 강의 IV · 연결 ⑥ — 연결 점검

## STEP 4

## 연결과 권한이 의도대로 됐는지 점검합니다

## ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 AI 도구 연동 검수 담당자야.

[맥락] 점검 범위 — 연결된 MCP 목록·상태 · 도구별 허용/차단 · 재무 분석에 필요한 연결 누락 여부

[요청] 현재 연결 상태와 권한을 종합 점검해줘.

[출력] [도구 | 연결 상태 | 권한 | 활용처 | 보완할 점] 표

[검증] 오류·누락은 원인과 해결 한 줄 → 조치 후 다시 점검한 결과

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

연결 완료 = Skill이 내부 자료(파일)와 외부 데이터(API)를 안전하게 쓸 준비 끝.



강의 V · 조합 ① — Command·Skill·MCP 언제 무엇을

 **Command**  
사용자가 /로 부르는 단축 절차

 **Skill**  
모델이 상황 보고 자동 호출

 **MCP**  
외부 도구·데이터 연결

| 업무 상황       | 활용                | 방식                  |
|-------------|-------------------|---------------------|
| 매주 같은 보고서   | 정해진 양식으로 초안 자동 생성 | Command (/주간보고 [팀]) |
| 문서를 표·엑셀로   | 자동 변환·집계          | Skill (xlsx)        |
| 최신 뉴스·시세 필요 | 실시간 조회            | MCP (웹 검색)          |
| 사내 DB 조회    | 고객·주문 데이터 가져오기    | MCP (데이터베이스)        |
| 반복 리포트 절차   | 절차를 자산화           | Skill (report)      |
| 팀에 배포·공유    | 묶어서 설치·공유         | plugin              |

핵심: 반복(Command·Skill)은 자산화, 외부(MCP)는 연결 — 셋을 조합하면 업무가 자동으로 흐릅니다.

## 강의 V · 조합 ② — 리포트 절차를 Skill로

전환

매주 반복하는 리포트 절차를 Skill 자산으로

## ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 업무 자동화 Skill 설계자야.

[맥락] 현재 절차 ① 재무·시장 데이터 수집 → ② 지표 계산·리스크 판정 → ③ 엑셀 표·차트 → ④ 요약 보고서 1장

[요청] 「주간 재무 리포트」 절차를 Skill로 전환해줘.

[출력] name 주간재무리포트 · description(호출 상황) · allowed-tools Read·xlsx·docx · 본문 4단계(입력·출력)  
산출물 output/재무리포트\_YYYYMMDD.xlsx · .docx

[검증] 한 번 실행해 4단계 산출물 생성 확인 · 같은 이름 파일이 있으면 덮어쓰기 전에 물어봐

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

반복 리포트가 Skill이 되면, 매주 "주간재무리포트 실행" 한 줄로 같은 품질의 보고서가 나옵니다.

## 강의 V · 조합 ③ — Skill에 데이터 소스 연결

연결

Skill이 스스로 내부·외부 데이터를 모으게 합니다

## ✓ 5요소 태그형 프롬프트 (그대로 복사해 입력)

[역할] 너는 업무 자동화 Skill 설계자야.

[맥락] 데이터 소스 — 내부: filesystem data/ 재무제표 · 외부: 웹 검색 최근 1주 뉴스 · 외부: 공개 API 기준금리·환율

[요청] 주간재무리포트 Skill이 실행될 때 데이터를 스스로 모으게 연결해줘.

[출력] SKILL.md ①단계에 3개 소스 수집 명시 · 실패 시 「해당 항목 없음」 · 모든 수치에 출처·기준일

[검증] [소스 | 수집 성공 | 건수 | 출처] 표 · 실패한 소스는 원인과 대안

1 Prompt

2 Context

3 Harness

4 Loop

5 Agentic

이 단계에서 특히 작동하는 엔지니어링 종류

이제 리포트 Skill = 내부 자료 + 외부 데이터를 스스로 모아 작성 — 다음 5교시(Multi-Agent)로 확장됩니다.

## 확장 ① · Yahoo 파이낸스 — 5종을 한 층씩

## ① Prompt

[역할] 재무 분석가 [요청] 삼성전자·SK하이닉스 최근 3개년 매출·영업이익·부채비율 정리 [출력] 지표 | 기업 | 값 | 연도 표

## ② Context

① + [근거 데이터] Yahoo 파이낸스 · 반도체 업종 평균과 비교 [표기] 수치마다 출처·조회 시점

## ③ Harness

② + [저장 형식] 엑셀 컬럼 기업·연도·지표·값·출처 고정 [스키마] DB financials 테이블

## ④ Loop

③ + [반복 조건] 누락 지표·이상치 [재수집] 원인 확인 후 다시 수집 [기록] 보정 내역 남기기

## ⑤ Agentic

④ + [단계 분담] 수집·정리·저장·요약을 단계로 실행 [보고] 단계마다 결과 보고

교육용 프롬프트 예시입니다(실제 투자 판단·자문 아님) · 각 층은 앞 층을 그대로 두고 태그 칸만 더합니다 — 다음 장에서 하나로 합칩니다.

## 확장 ② · 수집 → 엑셀 → DB 통합 프롬프트

## ✓ 태그형 통합 프롬프트 (수집 → 정리 → 저장 → 요약)

[역할] 너는 데이터 엔지니어야.

[맥락] 출처 Yahoo 파이낸스 · 기간 최근 3개년 · 대상 삼성전자(005930)·SK하이닉스(000660)  
지표 매출 · 영업이익 · 부채비율 · 영업이익률

[요청] 두 기업 재무 데이터를 수집 → 정리 → 저장 → 요약해줘.

[출력] ① 수집 — 실패 시 사유 기록

② 엑셀 재무비교.xlsx — 기업·연도·지표·값·출처·조회시점

③ sqlite finance.db 의 financials 테이블에 저장

④ 두 기업 비교 요약 3줄

[검증] 저장 건수와 누락 지표를 [기업 | 연도 | 누락지표] 표로 보고 · 누락은 원인 확인 후 재수집

[멈춤] 조회되지 않는 값은 추정하지 말고 「조회 실패」로 남겨줘

🌐 수집 Yahoo · MCP(웹/API)

📊 정리 엑셀 · Skill(xlsx)

🗄️ 저장 DB · MCP(파일/DB)

📄 요약 비교 리포트 3줄

## 엔지니어링 5종 모두 적용

① Prompt ② Context ③ Harness ④ Loop ⑤ Agentic

교육용 프롬프트 예시입니다(실제 투자 판단·자문 아님) · 수집→정리→저장→요약이 한 지시로 흐르도록 설계했습니다.

## [확장 ③] 내 반복 업무를 Skill로 만들고 연결하기

### 확장 ③ · 내 반복 업무를 Skill로 — 빈칸 [ ] · 총 15분

반복 업무 고르기 2분

빈칸 채우기 4분

Skill 생성 5분

연결·실행 3분

공유 1분

#### ① 내 주제를 빈칸에 적습니다

| 빈칸      | 무엇을 적나 · 예시                            |
|---------|----------------------------------------|
| [반복 업무] | 매번 같은 절차로 하는 일<br>예) 주간 보고서 작성         |
| [절차]    | 현재 순서 3~4단계<br>예) 수집 → 집계 → 표 → 요약     |
| [자료 소스] | 내부/외부 어디서 가져오나<br>예) data 폴더 엑셀 · 웹 검색 |
| [산출물]   | 최종 결과 형태<br>예) 엑셀 표 + 요약 1장            |

#### ② 아래를 복사해 [ ] 안만 바꿔 실행

[역할] 너는 업무 자동화 Skill 설계자야.

[맥락] 반복 업무는 [반복 업무], 현재 절차는 [절차], 자료는 [자료 소스]

[요청] 이 업무를 SKILL.md로 만들어줘.

[출력] name · description(언제 쓰는지) · allowed-tools(최소) · 본문 절차(단계별 입력·출력) · 산출물 [산출물]

[검증] 한 번 실행해 산출물이 나오는지 확인 · 모르는 값은 [ ]로 남기고 질문해줘

✓ 완료 기준 내 반복 업무의 SKILL.md가 만들어졌고, 한 번 실행해 산출물이 나왔다

※ 외부 데이터가 필요하면 "웹 검색으로 ○○도 가져와" 한 줄을 덧붙이면 MCP 연결까지 확장됩니다.

## Wrap-up · 오늘 배운 전부를 한 장에

## ① 실습 — 기준을 파일로

프로토타입(토큰·RQ) · VOC 답변 SKILL(조항 인용) · 검수 10

## ② Command · Skill 개념

사람이 부르는 Command · 모델이 부르는 Skill · 구조와 Scope

## ③ Skill 쓰고 만들기

가져다 쓰기 → 식별·정의·등록 → 태그형 SKILL.md로 설계

## ④ MCP 개념

표준 콘센트 · Tools·Resources·Prompts · TOP 10과 부서별 활용

## ⑤ 연결 · 권한

태그형 연결 지시 → 파일·API 연결 → 최소 권한 → 점검

## ⑥ 조합 · 확장

리포트 Skill + 데이터 소스 · Yahoo→엑셀→DB · 내 반복 업무 Skill

## 쉽게 이해하기

오늘은 TF 작업공간(폴더)에 "**자산(Skill)**과 **바깥 창구(MCP)**"를 놓았습니다. **다음 5교시** — 여러 에이전트가 이 자산·연결을 나눠 써 **Dashboard**를 완성합니다.